Assignment 4

Basic Info:

I used Claude and Copilot both accessed on my laptop.

Prompt:

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are

accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.

• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

• Write a recursive object-oriented program to find the solution to the five puzzle files.

• The output for each solution, should include:

o The puzzle file name (e.g., puzzle1.txt)

o The puzzle from the file printed out.

o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

Code:

Claude

```
// sudoku.cpp

// Build: g++ -std=c++17 -O2 -o sudoku sudoku.cpp

// Run:   ./sudoku puzzle1.txt puzzle2.txt ... (defaults to puzzle1..5.txt)


#include <array>

#include <fstream>

#include <iostream>

#include <string>
```

```
#include <vector>

class SudokuPuzzle {
public:
    static constexpr int N = 9;
    static constexpr int B = 3;

    explicit SudokuPuzzle(const std::string& fileName) : fileName_(fileName) {}

    // Reads 81 cells: digits 1-9 or '_' for blank. Whitespace is ignored.
    bool load() {
        std::ifstream in(fileName_);
        if (!in) return false;

        int count = 0;
        char ch;
        while (in.get(ch) && count < N * N) {
            if (ch >= '1' && ch <= '9')      grid_[count / N][count % N] = ch - '0';
            else if (ch == '_' || ch == '0') grid_[count / N][count % N] = 0;
            else continue;  // skip whitespace / newlines
            ++count;
        }
        return count == N * N;
    }

    void print(std::ostream& out) const { printGrid(out, grid_); }
```

```
    // Finds and prints every solution; returns how many were found.
    int solveAll(std::ostream& out) {
        solutionCount_ = 0;
        out_ = &out;
        search();
        return solutionCount_;
    }

    const std::string& fileName() const { return fileName_; }

private:
    using Grid = std::array<std::array<int, N>, N>;

    std::string fileName_;
    Grid grid_{};
    int solutionCount_ = 0;
    std::ostream* out_ = nullptr;

    // Can value v legally go at (row, col)?
    bool isCandidate(int row, int col, int v) const {
        for (int i = 0; i < N; ++i) {
            if (grid_[row][i] == v) return false;  // row
            if (grid_[i][col] == v) return false;  // column
        }
        int br = (row / B) * B, bc = (col / B) * B;
        for (int r = br; r < br + B; ++r)
            for (int c = bc; c < bc + B; ++c)
```

```
            if (grid_[r][c] == v) return false;  // block
    return true;
}

// Finds the first empty cell; returns false if the grid is full.
bool findEmpty(int& row, int& col) const {
    for (row = 0; row < N; ++row)
        for (col = 0; col < N; ++col)
            if (grid_[row][col] == 0) return true;
    return false;
}

// Recursive depth-first search with backtracking.
void search() {
    int row, col;
    if (!findEmpty(row, col)) {  // k == 81: a complete solution
        ++solutionCount_;
        *out_ << "Solution " << solutionCount_ << ":\n";
        printGrid(*out_, grid_);
        return;
    }
    for (int v = 1; v <= 9; ++v) {
        if (isCandidate(row, col, v)) {
            grid_[row][col] = v;
            search();
            grid_[row][col] = 0;  // backtrack
        }
```

```
        }
    }

    static void printGrid(std::ostream& out, const Grid& g) {
        for (int r = 0; r < N; ++r) {
            if (r > 0 && r % B == 0) out << "------+-------+------\n";
            for (int c = 0; c < N; ++c) {
                if (c > 0 && c % B == 0) out << "| ";
                out << (g[r][c] == 0 ? '_' : char('0' + g[r][c]));
                out << (c < N - 1 ? " " : "");
            }
            out << '\n';
        }
        out << '\n';
    }
};

int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    } else {
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".txt");
    }

    for (const auto& name : files) {
```

```
        SudokuPuzzle puzzle(name);
        std::cout << "==============================\n";
        std::cout << "Puzzle file: " << name << "\n\n";

        if (!puzzle.load()) {
            std::cout << "Could not read a valid puzzle from this file.\n\n";
            continue;
        }

        std::cout << "Puzzle:\n";
        puzzle.print(std::cout);

        int found = puzzle.solveAll(std::cout);
        if (found == 0) std::cout << "No solution found\n\n";
        else std::cout << found << " solution(s) found\n\n";
    }
    return 0;
}
```

Copilot

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuGrid {
private:
```

```
    int grid[9][9];

public:
    SudokuGrid() {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                grid[r][c] = 0;
    }

    bool loadFromFile(const std::string &filename) {
        std::ifstream in(filename);
        if (!in) return false;

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                char ch;
                if (!(in >> ch)) return false;
                if (ch == '_')
                    grid[r][c] = 0;
                else if (ch >= '1' && ch <= '9')
                    grid[r][c] = ch - '0';
                else
                    return false;
            }
        }
        return true;
    }
```

```
void print(std::ostream &out) const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            if (grid[r][c] == 0) out << "_";
            else out << grid[r][c];
            if (c < 8) out << " ";
        }
        out << "\n";
    }
}

int get(int r, int c) const { return grid[r][c]; }
void set(int r, int c, int val) { grid[r][c] = val; }

bool isEmpty(int r, int c) const { return grid[r][c] == 0; }

bool isValidPlacement(int r, int c, int val) const {
    // Row
    for (int col = 0; col < 9; ++col)
        if (grid[r][col] == val) return false;

    // Column
    for (int row = 0; row < 9; ++row)
        if (grid[row][c] == val) return false;

    // Block
```

```
        int br = (r / 3) * 3;
        int bc = (c / 3) * 3;
        for (int rr = 0; rr < 3; ++rr)
            for (int cc = 0; cc < 3; ++cc)
                if (grid[br + rr][bc + cc] == val) return false;

        return true;
    }

    bool isComplete() const {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (grid[r][c] == 0) return false;
        return true;
    }
};

class SudokuSolver {
private:
    SudokuGrid initial;
    std::vector<SudokuGrid> solutions;

    bool findEmptyCell(const SudokuGrid &g, int &row, int &col) const {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (g.isEmpty(r, c)) {
                    row = r;
```

```
                col = c;
                return true;
            }
        return false;
    }

    void solveRecursive(SudokuGrid &current) {
        int row, col;
        if (!findEmptyCell(current, row, col)) {
            // No empty cells: we have a complete solution
            if (current.isComplete())
                solutions.push_back(current);
            return;
        }

        // Try all candidates 1-9
        for (int val = 1; val <= 9; ++val) {
            if (current.isValidPlacement(row, col, val)) {
                current.set(row, col, val);
                solveRecursive(current);
                // Backtrack
                current.set(row, col, 0);
            }
        }
    }

public:
```

```
    SudokuSolver(const SudokuGrid &g) : initial(g) {}

    void solve() {
        solutions.clear();
        SudokuGrid working = initial;
        solveRecursive(working);
    }

    const std::vector<SudokuGrid> &getSolutions() const {
        return solutions;
    }
};

class SudokuApp {
public:
    void run() {
        for (int i = 1; i <= 5; ++i) {
            std::string filename = "puzzle" + std::to_string(i) + ".txt";
            SudokuGrid grid;
            std::cout << "Puzzle file: " << filename << "\n";

            if (!grid.loadFromFile(filename)) {
                std::cout << "Failed to load puzzle.\n\n";
                continue;
            }

            std::cout << "Original puzzle:\n";
```

```
            grid.print(std::cout);
            std::cout << "\n";

            SudokuSolver solver(grid);
            solver.solve();
            const auto &solutions = solver.getSolutions();

            if (solutions.empty()) {
                std::cout << "No solution found\n\n";
            } else {
                std::cout << "Found " << solutions.size() << " solution(s):\n";
                for (size_t idx = 0; idx < solutions.size(); ++idx) {
                    std::cout << "Solution " << (idx + 1) << ":\n";
                    solutions[idx].print(std::cout);
                    std::cout << "\n";
                }
            }
        }
    }
};

int main() {
    SudokuApp app;
    app.run();
    return 0;
}
```

Analysis:

Correctness:

They both run properly. However, they both don't check if the puzzles include conflicting clues or continue to read after 81 cells. Meaning larger puzzles wouldn't be read. Claude doesn't ensure the puzzle is valid beyond ensuring the file exists. Copilot doesn't accept a 0 as an empty cell.

Execution Time:

Each program has a similar execution time. They both use the same recursion with the same constraints as defined by Sudoku puzzles.

Space Complexity:

They have a similar space complexity when there is only one solution to a puzzle. However, if there are multiple solution Claude has a smaller space complexity. This is because Claude prints each solution as it's discovered. Copilot stores each solution then prints them all together. This means Copilots space increases with each new solution found while Claudes stays the same. Making Claude more efficient the more solutions there are.

Maintainability:

In terms of maintainability Copilot separates its responsibilities more. Meaning that changing the code later doesn't always require editing the entire program. It is also easier to implement in other programs. However, Claude uses a simpler design making it easier to understand and maintain. Claude also contains more comments explaining the code while Copilot includes virtually none.

Edits:

I will use the code generated by Claude. This is because the way Claude prints each puzzle is easier to read and Claude contains more comments. It has good execution time and beats Copilot when it comes to space complexity with multiple solutions. For my edits in terms of correctness, I will make the program validate the sudoku clues and validate the original puzzle. This ensures the program doesn't try to solve an invalid puzzle. This also improves execution time as time isn't wasted trying to solve a puzzle that never had a solution. I will also make the program choose an empty cell with the fewest possible numbers it could contain to start at. This improves execution time as it reduces the number of recursions needed to solve it. This is because the program doesn't have to try and retry a ton of possibilities it goes from what is most likely correct and base the rest of its solution off that.